# Ansible Variable Usage & Builtin Modules - Complete Examples

## Table of Contents



---

## Variable Syntax & Usage

### Basic Variable Reference

```
---
# Simple variable substitution
- name: Basic variables
  hosts: all
  vars:
    app_name: "myapp"
    app_port: 8080
    app_user: "appuser"

  tasks:
    - name: Display variable
      ansible.builtin.debug:
        msg: "Application {{ app_name }} runs on port {{ app_port }}"

    - name: Create user
      ansible.builtin.user:
        name: "{{ app_user }}"
        state: present
```

## Dictionary Variables

```
---
- name: Dictionary examples
  hosts: all
  vars:
    database:
      host: "db.example.com"
      port: 5432
      name: "production"
      user: "dbadmin"

    app_config:
      debug: false
      max_connections: 100
      timeout: 30

  tasks:
    - name: Access dict with dot notation
      ansible.builtin.debug:
        msg: "DB is {{ database.host }}:{{ database.port }}"

    - name: Access dict with bracket notation
      ansible.builtin.debug:
        msg: "Database name is {{ database['name'] }}"

    - name: Use in template
      ansible.builtin.template:
        src: "config.j2"
        dest: "/etc/app/config.yml"
        # Template content:
        # db_host: {{ database.host }}
        # db_port: {{ database.port }}
```

## List Variables

```
---
- name: List examples
  hosts: all
  vars:
    packages:
      - nginx
      - postgresql
```

```
      - redis

    users:
      - { name: "alice", uid: 1001, group: "admin" }
      - { name: "bob", uid: 1002, group: "users" }
      - { name: "charlie", uid: 1003, group: "users" }

  tasks:
    - name: Install packages from list
      ansible.builtin.apt:
        name: "{{ packages }}"
        state: present

    - name: Access list by index
      ansible.builtin.debug:
        msg: "First package is {{ packages[0] }}"

    - name: Loop over list
      ansible.builtin.user:
        name: "{{ item.name }}"
        uid: "{{ item.uid }}"
        group: "{{ item.group }}"
      loop: "{{ users }}"
```

## Environment Variables

```
---
- name: Environment variable examples
  hosts: all

  tasks:
    - name: Use environment variable
      ansible.builtin.debug:
        msg: "Home is {{ ansible_env.HOME }}"

    - name: Check PATH
      ansible.builtin.debug:
        msg: "PATH: {{ ansible_env.PATH }}"

    - name: Use custom environment
      ansible.builtin.shell: "echo $CUSTOM_VAR"
      environment:
        CUSTOM_VAR: "{{ app_name }}"
      register: result
```

```
    - name: Show result
      ansible.builtin.debug:
        msg: "{{ result.stdout }}"
```

## Registered Variables

```
---
- name: Register variable examples
  hosts: all

  tasks:
    - name: Get disk usage
      ansible.builtin.shell: "df -h /"
      register: disk_usage

    - name: Show disk usage output
      ansible.builtin.debug:
        msg: "{{ disk_usage.stdout }}"

    - name: Show return code
      ansible.builtin.debug:
        msg: "Return code: {{ disk_usage.rc }}"

    - name: Check if command changed anything
      ansible.builtin.debug:
        msg: "Changed: {{ disk_usage.changed }}"

    - name: Get file stats
      ansible.builtin.stat:
        path: "/etc/passwd"
      register: passwd_stat

    - name: Use stat results
      ansible.builtin.debug:
        msg: "File size: {{ passwd_stat.stat.size }} bytes"
      when: passwd_stat.stat.exists

    - name: Check user exists
      ansible.builtin.shell: "id {{ app_user }}"
      register: user_check
      ignore_errors: true

    - name: Conditional based on result
```

```
      ansible.builtin.debug:
        msg: "User exists"
      when: user_check.rc == 0
```

## Facts Variables

```
---
- name: Facts examples
  hosts: all
  gather_facts: yes

  tasks:
    - name: OS information
      ansible.builtin.debug:
        msg: "OS: {{ ansible_distribution }} {{ ansible_distribution_version }}"

    - name: Architecture
      ansible.builtin.debug:
        msg: "Architecture: {{ ansible_architecture }}"

    - name: Hostname
      ansible.builtin.debug:
        msg: "Hostname: {{ ansible_hostname }}, FQDN: {{ ansible_fqdn }}"

    - name: IP address
      ansible.builtin.debug:
        msg: "IP: {{ ansible_default_ipv4.address }}"

    - name: Memory
      ansible.builtin.debug:
        msg: "Total RAM: {{ ansible_memtotal_mb }} MB"

    - name: CPU cores
      ansible.builtin.debug:
        msg: "CPU cores: {{ ansible_processor_cores }}"

    - name: All network interfaces
      ansible.builtin.debug:
        msg: "Interfaces: {{ ansible_interfaces }}"

    - name: Mounted filesystems
      ansible.builtin.debug:
```

```
        msg: "Mount {{ item.mount }} - {{ item.size_total }}"
      loop: "{{ ansible_mounts }}"
```

## Magic Variables

```
---
- name: Magic variables examples
  hosts: all

  tasks:
    - name: Inventory hostname
      ansible.builtin.debug:
        msg: "Inventory name: {{ inventory_hostname }}"

    - name: Groups
      ansible.builtin.debug:
        msg: "I belong to: {{ group_names }}"

    - name: All groups
      ansible.builtin.debug:
        msg: "All groups: {{ groups }}"

    - name: Hosts in webservers group
      ansible.builtin.debug:
        msg: "Web servers: {{ groups['webservers'] }}"
      when: "'webservers' in groups"

    - name: Playbook directory
      ansible.builtin.debug:
        msg: "Playbook dir: {{ playbook_dir }}"

    - name: Ansible version
      ansible.builtin.debug:
        msg: "Ansible {{ ansible_version.full }}"

    - name: Access other host variables
      ansible.builtin.debug:
        msg: "Server1 IP: {{ hostvars['server1']['ansible_default_ipv4']['address'] }}"
      when: "'server1' in hostvars"
```

### Variable Precedence Examples

```
---
- name: Variable precedence demo
  hosts: all
  vars:
    my_var: "playbook_vars"
  vars_files:
    - vars/main.yml  # Contains: my_var: "vars_file"

  tasks:
    - name: Show variable (vars_files wins over vars)
      ansible.builtin.debug:
        msg: "{{ my_var }}"

    - name: Set fact (higher precedence)
      ansible.builtin.set_fact:
        my_var: "set_fact_value"

    - name: Show updated variable
      ansible.builtin.debug:
        msg: "{{ my_var }}"

    # Command line -e my_var="extra_vars" would override everything
```

---

## Variable Manipulation & Filters

### String Filters

```
---
- name: String manipulation
  hosts: all
  vars:
    app_name: "MyApplication"
    path: "/opt/myapp/config"

  tasks:
    - name: Lowercase
      ansible.builtin.debug:
        msg: "{{ app_name | lower }}"  # myapplication
```

```yaml
- name: Uppercase
  ansible.builtin.debug:
    msg: "{{ app_name | upper }}"  # MYAPPLICATION

- name: Capitalize
  ansible.builtin.debug:
    msg: "{{ app_name | capitalize }}"  # Myapplication

- name: Replace text
  ansible.builtin.debug:
    msg: "{{ app_name | replace('Application', 'Service') }}"

- name: String length
  ansible.builtin.debug:
    msg: "Length: {{ app_name | length }}"

- name: Basename
  ansible.builtin.debug:
    msg: "{{ path | basename }}"  # config

- name: Dirname
  ansible.builtin.debug:
    msg: "{{ path | dirname }}"  # /opt/myapp

- name: Split string
  ansible.builtin.debug:
    msg: "{{ path | split('/') }}"

- name: Join list
  ansible.builtin.debug:
    msg: "{{ ['a', 'b', 'c'] | join('-') }}"  # a-b-c

- name: Default value if undefined
  ansible.builtin.debug:
    msg: "{{ undefined_var | default('fallback') }}"

- name: Trim whitespace
  ansible.builtin.debug:
    msg: "{{ '  text  ' | trim }}"
```

## List Filters

```
---
- name: List manipulation
  hosts: all
  vars:
    numbers: [1, 2, 3, 4, 5]
    servers:
      - { name: "web1", role: "web" }
      - { name: "db1", role: "database" }
      - { name: "web2", role: "web" }

  tasks:
    - name: First item
      ansible.builtin.debug:
        msg: "{{ numbers | first }}"  # 1

    - name: Last item
      ansible.builtin.debug:
        msg: "{{ numbers | last }}"  # 5

    - name: Min value
      ansible.builtin.debug:
        msg: "{{ numbers | min }}"

    - name: Max value
      ansible.builtin.debug:
        msg: "{{ numbers | max }}"

    - name: Sum values
      ansible.builtin.debug:
        msg: "{{ numbers | sum }}"

    - name: Unique items
      ansible.builtin.debug:
        msg: "{{ [1, 2, 2, 3] | unique }}"

    - name: Sort list
      ansible.builtin.debug:
        msg: "{{ [3, 1, 2] | sort }}"

    - name: Reverse list
      ansible.builtin.debug:
        msg: "{{ numbers | reverse }}"
```

```
    - name: Select attribute from list
      ansible.builtin.debug:
        msg: "{{ servers | map(attribute='name') | list }}"

    - name: Filter by attribute
      ansible.builtin.debug:
        msg: "{{ servers | selectattr('role', 'equalto', 'web') | list }}"

    - name: Reject by attribute
      ansible.builtin.debug:
        msg: "{{ servers | rejectattr('role', 'equalto', 'web') | list }}"
```

## Dictionary Filters

```
---
- name: Dictionary manipulation
  hosts: all
  vars:
    config:
      host: "localhost"
      port: 8080
      debug: true

  tasks:
    - name: Get keys
      ansible.builtin.debug:
        msg: "{{ config | dict2items | map(attribute='key') | list }}"

    - name: Get values
      ansible.builtin.debug:
        msg: "{{ config | dict2items | map(attribute='value') | list }}"

    - name: Combine dictionaries
      ansible.builtin.debug:
        msg: "{{ {'a': 1} | combine({'b': 2}) }}"

    - name: Extract keys
      ansible.builtin.debug:
        msg: "{{ config.keys() | list }}"

    - name: Extract values
      ansible.builtin.debug:
        msg: "{{ config.values() | list }}"
```

```
    - name: Convert dict to items
      ansible.builtin.debug:
        msg: "{{ config | dict2items }}"

    - name: Items to dict
      ansible.builtin.debug:
        msg: "{{ [{'key': 'a', 'value': 1}] | items2dict }}"
```

## Type Conversion

```
---
- name: Type conversion
  hosts: all
  vars:
    str_number: "42"
    number: 42

  tasks:
    - name: String to int
      ansible.builtin.debug:
        msg: "{{ str_number | int }}"

    - name: String to float
      ansible.builtin.debug:
        msg: "{{ '3.14' | float }}"

    - name: Int to string
      ansible.builtin.debug:
        msg: "{{ number | string }}"

    - name: To boolean
      ansible.builtin.debug:
        msg: "{{ 'yes' | bool }}"

    - name: To JSON
      ansible.builtin.debug:
        msg: "{{ {'key': 'value'} | to_json }}"

    - name: To YAML
      ansible.builtin.debug:
        msg: "{{ {'key': 'value'} | to_yaml }}"

    - name: From JSON
```

```
      ansible.builtin.debug:
        msg: "{{ '{\"key\": \"value\"}' | from_json }}"
```

## Conditionals in Variables

```
---
- name: Conditional variables
  hosts: all

  tasks:
    - name: Ternary operator
      ansible.builtin.debug:
        msg: "{{ (ansible_distribution == 'Ubuntu') | ternary('apt', 'yum') }}"

    - name: Default with condition
      ansible.builtin.debug:
        msg: "{{ my_var | default('default_value') }}"

    - name: Mandatory (fail if undefined)
      ansible.builtin.debug:
        msg: "{{ required_var | mandatory }}"
      ignore_errors: yes
```

## Date/Time Filters

```
---
- name: Date and time
  hosts: all

  tasks:
    - name: Current time
      ansible.builtin.debug:
        msg: "{{ ansible_date_time.date }}"

    - name: ISO time
      ansible.builtin.debug:
        msg: "{{ ansible_date_time.iso8601 }}"

    - name: Epoch time
      ansible.builtin.debug:
        msg: "{{ ansible_date_time.epoch }}"
```

```
    - name: Format date
      ansible.builtin.debug:
        msg: "{{ '%Y-%m-%d' | strftime }}"
```

## Math Operations

```
---
- name: Math operations
  hosts: all
  vars:
    num1: 10
    num2: 3

  tasks:
    - name: Addition
      ansible.builtin.debug:
        msg: "{{ num1 + num2 }}"

    - name: Multiplication
      ansible.builtin.debug:
        msg: "{{ num1 * num2 }}"

    - name: Division
      ansible.builtin.debug:
        msg: "{{ num1 / num2 }}"

    - name: Modulo
      ansible.builtin.debug:
        msg: "{{ num1 % num2 }}"

    - name: Power
      ansible.builtin.debug:
        msg: "{{ num1 ** 2 }}"

    - name: Absolute value
      ansible.builtin.debug:
        msg: "{{ -5 | abs }}"

    - name: Round
      ansible.builtin.debug:
        msg: "{{ 3.7 | round }}"
```

## Network Filters

```
---
- name: Network operations
  hosts: all
  vars:
    ip_addr: "192.168.1.10"
    cidr: "192.168.1.0/24"

  tasks:
    - name: IP address info
      ansible.builtin.debug:
        msg: "{{ ip_addr | ipaddr }}"

    - name: Network address
      ansible.builtin.debug:
        msg: "{{ cidr | ipaddr('network') }}"

    - name: Netmask
      ansible.builtin.debug:
        msg: "{{ cidr | ipaddr('netmask') }}"

    - name: Broadcast
      ansible.builtin.debug:
        msg: "{{ cidr | ipaddr('broadcast') }}"
```

## Hash/Encryption Filters

```
---
- name: Hash and encryption
  hosts: all
  vars:
    password: "secret123"

  tasks:
    - name: MD5 hash
      ansible.builtin.debug:
        msg: "{{ password | hash('md5') }}"

    - name: SHA256 hash
      ansible.builtin.debug:
        msg: "{{ password | hash('sha256') }}"
```

```
    - name: SHA512 password hash
      ansible.builtin.debug:
        msg: "{{ password | password_hash('sha512') }}"

    - name: Base64 encode
      ansible.builtin.debug:
        msg: "{{ password | b64encode }}"

    - name: Base64 decode
      ansible.builtin.debug:
        msg: "{{ 'c2VjcmV0MTIz' | b64decode }}"
```

## URL Filters

```
---
- name: URL operations
  hosts: all
  vars:
    url: "https://example.com/path?key=value"
    text: "Hello World!"

  tasks:
    - name: URL encode
      ansible.builtin.debug:
        msg: "{{ text | urlencode }}"

    - name: Extract hostname
      ansible.builtin.debug:
        msg: "{{ url | urlsplit('hostname') }}"

    - name: Extract path
      ansible.builtin.debug:
        msg: "{{ url | urlsplit('path') }}"
```

## Regex Filters

```
---
- name: Regex operations
  hosts: all
  vars:
    text: "Error: Connection failed on port 8080"
```

```
  tasks:
    - name: Regex search
      ansible.builtin.debug:
        msg: "{{ text | regex_search('port (\\d+)') }}"

    - name: Regex replace
      ansible.builtin.debug:
        msg: "{{ text | regex_replace('\\d+', 'XXXX') }}"

    - name: Regex findall
      ansible.builtin.debug:
        msg: "{{ text | regex_findall('\\w+') }}"
```

---

## Ansible Builtin Modules - Complete Examples

### ansible.builtin.debug

```
---
- name: Debug examples
  hosts: all
  vars:
    app_version: "2.1.0"

  tasks:
    - name: Simple message
      ansible.builtin.debug:
        msg: "Hello World"

    - name: Variable display
      ansible.builtin.debug:
        msg: "App version: {{ app_version }}"

    - name: Multiple variables
      ansible.builtin.debug:
        msg: "OS: {{ ansible_distribution }}, Version: {{ ansible_distribution_version }}"

    - name: Display variable contents
      ansible.builtin.debug:
        var: ansible_default_ipv4

    - name: Conditional debug
```

```
      ansible.builtin.debug:
        msg: "This is Ubuntu"
      when: ansible_distribution == "Ubuntu"

    - name: Debug with verbosity control
      ansible.builtin.debug:
        msg: "This only shows with -vv"
        verbosity: 2
```

## ansible.builtin.set_fact

```
---
- name: Set fact examples
  hosts: all

  tasks:
    - name: Set simple fact
      ansible.builtin.set_fact:
        my_custom_var: "custom_value"

    - name: Set multiple facts
      ansible.builtin.set_fact:
        db_host: "db.example.com"
        db_port: 5432
        db_name: "production"

    - name: Set fact from calculation
      ansible.builtin.set_fact:
        total_memory_gb: "{{ (ansible_memtotal_mb / 1024) | round }}"

    - name: Set fact from command result
      ansible.builtin.shell: "uname -r"
      register: kernel_version

    - name: Store kernel version as fact
      ansible.builtin.set_fact:
        current_kernel: "{{ kernel_version.stdout }}"

    - name: Conditional fact
      ansible.builtin.set_fact:
        pkg_manager: "{{ 'apt' if ansible_os_family == 'Debian' else 'yum' }}"

    - name: Set complex fact
      ansible.builtin.set_fact:
```

```
        app_config:
          name: "myapp"
          port: 8080
          debug: false

    - name: Use the facts
      ansible.builtin.debug:
        msg: "DB: {{ db_host }}:{{ db_port }}, Kernel: {{ current_kernel }}"
```

## ansible.builtin.command

```
---
- name: Command examples
  hosts: all

  tasks:
    - name: Run simple command
      ansible.builtin.command: "ls -la /tmp"
      register: tmp_contents

    - name: Show command output
      ansible.builtin.debug:
        msg: "{{ tmp_contents.stdout_lines }}"

    - name: Command with arguments
      ansible.builtin.command:
        cmd: "cat /etc/hostname"
      register: hostname_content

    - name: Command with working directory
      ansible.builtin.command:
        cmd: "ls -la"
        chdir: "/var/log"
      register: log_dir

    - name: Command with creates (idempotent)
      ansible.builtin.command:
        cmd: "touch /tmp/myfile"
        creates: "/tmp/myfile"

    - name: Command with removes (idempotent)
      ansible.builtin.command:
        cmd: "rm /tmp/oldfile"
        removes: "/tmp/oldfile"
```

```
    - name: Command with environment
      ansible.builtin.command: "printenv CUSTOM_VAR"
      environment:
        CUSTOM_VAR: "{{ app_name }}"
      register: env_output

    - name: Command with changed_when
      ansible.builtin.command: "echo 'test'"
      changed_when: false

    - name: Command with failed_when
      ansible.builtin.command: "grep 'pattern' /etc/hosts"
      register: grep_result
      failed_when: grep_result.rc not in [0, 1]
```

## ansible.builtin.shell

```
---
- name: Shell examples
  hosts: all

  tasks:
    - name: Run shell command with pipes
      ansible.builtin.shell: "ps aux | grep nginx | wc -l"
      register: nginx_count

    - name: Shell with redirection
      ansible.builtin.shell: "echo 'test' > /tmp/output.txt"

    - name: Shell with wildcards
      ansible.builtin.shell: "ls /var/log/*.log"
      register: log_files

    - name: Multi-line shell script
      ansible.builtin.shell: |
        if [ -f /etc/redhat-release ]; then
          echo "RedHat"
        else
          echo "Other"
        fi
      register: os_check

    - name: Shell with specific shell
```

```
      ansible.builtin.shell:
        cmd: "echo $SHELL"
        executable: "/bin/bash"
      register: shell_output

    - name: Shell with variables
      ansible.builtin.shell: "echo {{ ansible_hostname }}"
      register: hostname_echo
```

## ansible.builtin.copy

```
---
- name: Copy examples
  hosts: all

  tasks:
    - name: Copy file from local to remote
      ansible.builtin.copy:
        src: "/local/path/file.txt"
        dest: "/remote/path/file.txt"
        owner: "root"
        group: "root"
        mode: "0644"

    - name: Copy with backup
      ansible.builtin.copy:
        src: "{{ playbook_dir }}/files/config.yml"
        dest: "/etc/app/config.yml"
        backup: yes

    - name: Copy with content inline
      ansible.builtin.copy:
        content: |
          server {
            listen {{ app_port }};
            server_name {{ app_domain }};
          }
        dest: "/etc/nginx/sites-available/{{ app_name }}"
        mode: "0644"

    - name: Copy directory
      ansible.builtin.copy:
        src: "/local/dir/"
        dest: "/remote/dir/"
```

```
        mode: "0755"

    - name: Copy with validation
      ansible.builtin.copy:
        src: "nginx.conf"
        dest: "/etc/nginx/nginx.conf"
        validate: "nginx -t -c %s"

    - name: Copy and set SELinux context
      ansible.builtin.copy:
        src: "webapp.conf"
        dest: "/etc/httpd/conf.d/webapp.conf"
        setype: "httpd_config_t"

    - name: Force overwrite
      ansible.builtin.copy:
        src: "file.txt"
        dest: "/tmp/file.txt"
        force: yes

    - name: Remote to remote copy
      ansible.builtin.copy:
        src: "/tmp/source.txt"
        dest: "/tmp/dest.txt"
        remote_src: yes
```

## ansible.builtin.template

```
---
- name: Template examples
  hosts: all
  vars:
    db_host: "localhost"
    db_port: 5432
    app_workers: 4

  tasks:
    - name: Deploy template
      ansible.builtin.template:
        src: "{{ playbook_dir }}/templates/config.j2"
        dest: "/etc/app/config.yml"
        owner: "app"
        group: "app"
        mode: "0640"
```

```
    # templates/config.j2:
    # database:
    #   host: {{ db_host }}
    #   port: {{ db_port }}
    # workers: {{ app_workers }}

    - name: Template with validation
      ansible.builtin.template:
        src: "nginx.conf.j2"
        dest: "/etc/nginx/nginx.conf"
        validate: "nginx -t -c %s"

    - name: Template with backup
      ansible.builtin.template:
        src: "app.conf.j2"
        dest: "/etc/app.conf"
        backup: yes

    - name: Template with variables
      ansible.builtin.template:
        src: "script.sh.j2"
        dest: "/usr/local/bin/script.sh"
        mode: "0755"
      vars:
        script_timeout: 300
        script_retries: 3
```

## ansible.builtin.file

```
---
- name: File examples
  hosts: all

  tasks:
    - name: Create directory
      ansible.builtin.file:
        path: "/opt/myapp"
        state: directory
        mode: "0755"
        owner: "app"
        group: "app"

    - name: Create nested directories
```

```
    ansible.builtin.file:
      path: "/var/log/myapp/archives"
      state: directory
      mode: "0750"
      recurse: yes

  - name: Create file
    ansible.builtin.file:
      path: "/tmp/myfile.txt"
      state: touch
      mode: "0644"

  - name: Delete file
    ansible.builtin.file:
      path: "/tmp/oldfile"
      state: absent

  - name: Delete directory recursively
    ansible.builtin.file:
      path: "/tmp/old_dir"
      state: absent

  - name: Create symbolic link
    ansible.builtin.file:
      src: "/opt/myapp/current"
      dest: "/opt/myapp/releases/{{ app_version }}"
      state: link

  - name: Change permissions
    ansible.builtin.file:
      path: "/etc/app/secret.conf"
      mode: "0600"
      owner: "root"
      group: "root"

  - name: Set SELinux context
    ansible.builtin.file:
      path: "/var/www/html"
      setype: "httpd_sys_content_t"
      recurse: yes

  - name: Change ownership recursively
    ansible.builtin.file:
      path: "/opt/myapp"
      owner: "app"
```

```
        group: "app"
        recurse: yes
```

## ansible.builtin.lineinfile

```
---
- name: Lineinfile examples
  hosts: all

  tasks:
    - name: Ensure line exists
      ansible.builtin.lineinfile:
        path: "/etc/hosts"
        line: "192.168.1.10 myserver.local"
        state: present

    - name: Replace line with regex
      ansible.builtin.lineinfile:
        path: "/etc/ssh/sshd_config"
        regexp: "^#?PermitRootLogin"
        line: "PermitRootLogin no"

    - name: Add line after pattern
      ansible.builtin.lineinfile:
        path: "/etc/fstab"
        insertafter: "^# Custom mounts"
        line: "/dev/sdb1 /data ext4 defaults 0 0"

    - name: Add line before pattern
      ansible.builtin.lineinfile:
        path: "/etc/profile"
        insertbefore: "^export PATH"
        line: "export JAVA_HOME=/usr/lib/jvm/java-11"

    - name: Add line at beginning
      ansible.builtin.lineinfile:
        path: "/etc/rc.local"
        insertbefore: "BOF"
        line: "#!/bin/bash"

    - name: Add line at end
      ansible.builtin.lineinfile:
        path: "/etc/environment"
        insertafter: "EOF"
```

```
        line: "MY_VAR={{ my_value }}"

    - name: Remove line
      ansible.builtin.lineinfile:
        path: "/etc/hosts"
        regexp: "^127.0.1.1"
        state: absent

    - name: With backup
      ansible.builtin.lineinfile:
        path: "/etc/config.conf"
        regexp: "^timeout="
        line: "timeout={{ timeout_value }}"
        backup: yes

    - name: Validate after change
      ansible.builtin.lineinfile:
        path: "/etc/sudoers"
        regexp: "^%wheel"
        line: "%wheel ALL=(ALL) ALL"
        validate: "visudo -cf %s"
```

## ansible.builtin.blockinfile

```
---
- name: Blockinfile examples
  hosts: all

  tasks:
    - name: Insert configuration block
      ansible.builtin.blockinfile:
        path: "/etc/ssh/sshd_config"
        block: |
          # Custom SSH settings
          ClientAliveInterval 300
          ClientAliveCountMax 2
          MaxAuthTries 3

    - name: Insert with custom markers
      ansible.builtin.blockinfile:
        path: "/etc/hosts"
        marker: "# {mark} ANSIBLE MANAGED HOSTS"
        block: |
          192.168.1.10 web1
```

```
          192.168.1.11 web2
          192.168.1.12 db1

    - name: Insert after pattern
      ansible.builtin.blockinfile:
        path: "/etc/nginx/nginx.conf"
        insertafter: "http {"
        block: |
          client_max_body_size 100M;
          keepalive_timeout 65;

    - name: Remove block
      ansible.builtin.blockinfile:
        path: "/etc/config"
        marker: "# {mark} OLD CONFIG"
        state: absent

    - name: Block with variables
      ansible.builtin.blockinfile:
        path: "/etc/app/config"
        block: |
          DB_HOST={{ db_host }}
          DB_PORT={{ db_port }}
          DB_NAME={{ db_name }}
```

## ansible.builtin.replace

```
---
- name: Replace examples
  hosts: all

  tasks:
    - name: Simple replace
      ansible.builtin.replace:
        path: "/etc/config.conf"
        regexp: "old_value"
        replace: "new_value"

    - name: Replace with regex groups
      ansible.builtin.replace:
        path: "/etc/hosts"
        regexp: "^(127\\.0\\.0\\.1)\\s+.*"
        replace: "\\1 localhost"
```

```
    - name: Replace with backup
      ansible.builtin.replace:
        path: "/etc/important.conf"
        regexp: "timeout=\\d+"
        replace: "timeout={{ timeout_value }}"
        backup: yes

    - name: Replace before/after markers
      ansible.builtin.replace:
        path: "/etc/config"
        after: "# START SECTION"
        before: "# END SECTION"
        regexp: "enabled=false"
        replace: "enabled=true"
```

## ansible.builtin.stat

```
---
- name: Stat examples
  hosts: all

  tasks:
    - name: Get file information
      ansible.builtin.stat:
        path: "/etc/passwd"
      register: passwd_stat

    - name: Check file exists
      ansible.builtin.debug:
        msg: "File exists"
      when: passwd_stat.stat.exists

    - name: Check if directory
      ansible.builtin.debug:
        msg: "Is directory"
      when: passwd_stat.stat.isdir

    - name: Check if regular file
      ansible.builtin.debug:
        msg: "Is file"
      when: passwd_stat.stat.isreg

    - name: Check if link
      ansible.builtin.debug:
```

```
        msg: "Is symlink"
      when: passwd_stat.stat.islnk

    - name: Show file size
      ansible.builtin.debug:
        msg: "Size: {{ passwd_stat.stat.size }} bytes"

    - name: Check file age
      ansible.builtin.stat:
        path: "/var/log/app.log"
      register: log_stat

    - name: Delete if older than 7 days
      ansible.builtin.file:
        path: "/var/log/app.log"
        state: absent
      when: >
        (ansible_date_time.epoch | int) - log_stat.stat.mtime > 604800

    - name: Get checksum
      ansible.builtin.stat:
        path: "/etc/config"
        checksum_algorithm: "sha256"
      register: config_stat

    - name: Show checksum
      ansible.builtin.debug:
        msg: "SHA256: {{ config_stat.stat.checksum }}"
```

## ansible.builtin.find

```
---
- name: Find examples
  hosts: all

  tasks:
    - name: Find all log files
      ansible.builtin.find:
        paths: "/var/log"
        patterns: "*.log"
      register: log_files

    - name: Find files recursively
      ansible.builtin.find:
```

```
      paths: "/opt/app"
      patterns: "*.conf"
      recurse: yes
    register: conf_files

  - name: Find by age
    ansible.builtin.find:
      paths: "/tmp"
      age: "7d"
      age_stamp: "mtime"
    register: old_files

  - name: Delete old files
    ansible.builtin.file:
      path: "{{ item.path }}"
      state: absent
    loop: "{{ old_files.files }}"

  - name: Find by size
    ansible.builtin.find:
      paths: "/var/log"
      size: "100m"
    register: large_files

  - name: Find directories
    ansible.builtin.find:
      paths: "/opt"
      file_type: "directory"
    register: directories

  - name: Find with multiple patterns
    ansible.builtin.find:
      paths: "/etc"
      patterns: "*.conf,*.cfg"
    register: config_files

  - name: Find and get checksums
    ansible.builtin.find:
      paths: "/etc/ssl/certs"
      patterns: "*.pem"
      get_checksum: yes
    register: cert_files
```

## ansible.builtin.get_url

```
---
- name: Get_url examples
  hosts: all

  tasks:
    - name: Download file
      ansible.builtin.get_url:
        url: "https://example.com/file.tar.gz"
        dest: "/tmp/file.tar.gz"
        mode: "0644"

    - name: Download with checksum validation
      ansible.builtin.get_url:
        url: "https://example.com/package.deb"
        dest: "/tmp/package.deb"
        checksum: "sha256:abc123..."

    - name: Download with basic auth
      ansible.builtin.get_url:
        url: "https://secure.example.com/file"
        dest: "/tmp/file"
        url_username: "{{ api_user }}"
        url_password: "{{ api_pass }}"

    - name: Download with custom headers
      ansible.builtin.get_url:
        url: "https://api.example.com/download"
        dest: "/tmp/download"
        headers:
          Authorization: "Bearer {{ api_token }}"

    - name: Download with timeout
      ansible.builtin.get_url:
        url: "https://slow.example.com/bigfile"
        dest: "/tmp/bigfile"
        timeout: 300

    - name: Download and validate SSL
      ansible.builtin.get_url:
        url: "https://secure.example.com/file"
        dest: "/tmp/file"
        validate_certs: yes
```

```
    - name: Download if newer
      ansible.builtin.get_url:
        url: "https://example.com/latest.zip"
        dest: "/tmp/latest.zip"
        force: no
```

### ansible.builtin.unarchive

```
---
- name: Unarchive examples
  hosts: all

  tasks:
    - name: Extract tar.gz
      ansible.builtin.unarchive:
        src: "/tmp/archive.tar.gz"
        dest: "/opt/app"
        remote_src: yes

    - name: Extract from local file
      ansible.builtin.unarchive:
        src: "{{ playbook_dir }}/files/app.zip"
        dest: "/opt/app"

    - name: Extract specific files
      ansible.builtin.unarchive:
        src: "/tmp/backup.tar.gz"
        dest: "/restore"
        include:
          - "data/*"
          - "config.yml"
        remote_src: yes

    - name: Extract with ownership
      ansible.builtin.unarchive:
        src: "/tmp/app.tar.gz"
        dest: "/opt/app"
        owner: "app"
        group: "app"
        remote_src: yes

    - name: Extract and create directory
      ansible.builtin.unarchive:
        src: "/tmp/files.zip"
```

```
        dest: "/opt/extracted"
        creates: "/opt/extracted/marker_file"
        remote_src: yes
```

## ansible.builtin.archive

```
---
- name: Archive examples
  hosts: all

  tasks:
    - name: Create tar.gz archive
      ansible.builtin.archive:
        path: "/opt/app"
        dest: "/backup/app-{{ ansible_date_time.date }}.tar.gz"
        format: "gz"

    - name: Create zip archive
      ansible.builtin.archive:
        path:
          - "/etc/config"
          - "/var/log/app.log"
        dest: "/backup/config.zip"
        format: "zip"

    - name: Archive with exclusions
      ansible.builtin.archive:
        path: "/opt/app"
        dest: "/backup/app.tar.gz"
        exclude_path:
          - "/opt/app/cache"
          - "/opt/app/*.log"

    - name: Create bz2 archive
      ansible.builtin.archive:
        path: "/var/log"
        dest: "/backup/logs.tar.bz2"
        format: "bz2"
```

## ansible.builtin.synchronize

```
---
- name: Synchronize examples
  hosts: all

  tasks:
    - name: Sync directory
      ansible.builtin.synchronize:
        src: "/local/src/"
        dest: "/remote/dest/"

    - name: Sync with delete
      ansible.builtin.synchronize:
        src: "/source/"
        dest: "/dest/"
        delete: yes

    - name: Sync with excludes
      ansible.builtin.synchronize:
        src: "/app/"
        dest: "/backup/app/"
        rsync_opts:
          - "--exclude=*.log"
          - "--exclude=cache/"

    - name: Pull from remote
      ansible.builtin.synchronize:
        mode: "pull"
        src: "/remote/data/"
        dest: "/local/backup/"

    - name: Sync with checksum
      ansible.builtin.synchronize:
        src: "/source/"
        dest: "/dest/"
        checksum: yes
```

## ansible.builtin.user

```
---
- name: User examples
  hosts: all
```

```yaml
tasks:
  - name: Create user
    ansible.builtin.user:
      name: "{{ app_user }}"
      state: present
      shell: "/bin/bash"
      home: "/home/{{ app_user }}"

  - name: Create user with specific UID
    ansible.builtin.user:
      name: "webapp"
      uid: 1500
      group: "webapp"
      state: present

  - name: Create system user
    ansible.builtin.user:
      name: "service_account"
      system: yes
      shell: "/usr/sbin/nologin"
      create_home: no

  - name: Add user to groups
    ansible.builtin.user:
      name: "{{ app_user }}"
      groups: "sudo,docker"
      append: yes

  - name: Set user password
    ansible.builtin.user:
      name: "{{ app_user }}"
      password: "{{ 'mypassword' | password_hash('sha512') }}"
      update_password: "always"

  - name: Create user with SSH key
    ansible.builtin.user:
      name: "deploy"
      generate_ssh_key: yes
      ssh_key_bits: 4096
      ssh_key_file: ".ssh/id_rsa"

  - name: Lock user account
    ansible.builtin.user:
      name: "olduser"
      password_lock: yes
```

```
    - name: Set expiration date
      ansible.builtin.user:
        name: "tempuser"
        expires: "{{ (ansible_date_time.epoch | int) + 2592000 }}"

    - name: Remove user
      ansible.builtin.user:
        name: "olduser"
        state: absent
        remove: yes
```

## ansible.builtin.group

```
---
- name: Group examples
  hosts: all

  tasks:
    - name: Create group
      ansible.builtin.group:
        name: "{{ app_group }}"
        state: present

    - name: Create group with GID
      ansible.builtin.group:
        name: "webapp"
        gid: 1500
        state: present

    - name: Create system group
      ansible.builtin.group:
        name: "sysgroup"
        system: yes
        state: present

    - name: Remove group
      ansible.builtin.group:
        name: "oldgroup"
        state: absent
```

## ansible.builtin.service

```
---
- name: Service examples
  hosts: all

  tasks:
    - name: Start service
      ansible.builtin.service:
        name: "nginx"
        state: started

    - name: Stop service
      ansible.builtin.service:
        name: "apache2"
        state: stopped

    - name: Restart service
      ansible.builtin.service:
        name: "{{ app_service }}"
        state: restarted

    - name: Reload service
      ansible.builtin.service:
        name: "nginx"
        state: reloaded

    - name: Enable service
      ansible.builtin.service:
        name: "{{ app_service }}"
        enabled: yes

    - name: Disable service
      ansible.builtin.service:
        name: "oldservice"
        enabled: no

    - name: Start and enable
      ansible.builtin.service:
        name: "postgresql"
        state: started
        enabled: yes

    - name: Restart if enabled
      ansible.builtin.service:
```

```
        name: "nginx"
        state: restarted
      when: nginx_enabled | default(true)
```

## ansible.builtin.systemd

```
---
- name: Systemd examples
  hosts: all

  tasks:
    - name: Start systemd service
      ansible.builtin.systemd:
        name: "myapp.service"
        state: started

    - name: Enable and start
      ansible.builtin.systemd:
        name: "{{ app_service }}"
        state: started
        enabled: yes
        daemon_reload: yes

    - name: Reload systemd daemon
      ansible.builtin.systemd:
        daemon_reload: yes

    - name: Mask service
      ansible.builtin.systemd:
        name: "unwanted.service"
        masked: yes

    - name: Unmask service
      ansible.builtin.systemd:
        name: "service.service"
        masked: no
```

## ansible.builtin.package

```
---
- name: Package examples
  hosts: all
```

```
  tasks:
    - name: Install package (auto-detect manager)
      ansible.builtin.package:
        name: "{{ item }}"
        state: present
      loop:
        - git
        - curl
        - vim

    - name: Install specific version
      ansible.builtin.package:
        name: "nginx=1.18.*"
        state: present

    - name: Remove package
      ansible.builtin.package:
        name: "oldpackage"
        state: absent

    - name: Install latest
      ansible.builtin.package:
        name: "python3"
        state: latest
```

## ansible.builtin.apt

```
---
- name: APT examples
  hosts: ubuntu

  tasks:
    - name: Update cache
      ansible.builtin.apt:
        update_cache: yes
        cache_valid_time: 3600

    - name: Install packages
      ansible.builtin.apt:
        name:
          - nginx
          - postgresql
          - redis-server
```

```
        state: present

    - name: Install specific version
      ansible.builtin.apt:
        name: "docker-ce=5:20.10.7~3-0~ubuntu-focal"
        state: present

    - name: Remove package
      ansible.builtin.apt:
        name: "apache2"
        state: absent

    - name: Remove with purge
      ansible.builtin.apt:
        name: "oldpackage"
        state: absent
        purge: yes

    - name: Install .deb file
      ansible.builtin.apt:
        deb: "/tmp/package.deb"

    - name: Upgrade all packages
      ansible.builtin.apt:
        upgrade: "dist"

    - name: Autoremove
      ansible.builtin.apt:
        autoremove: yes

    - name: Install with recommends
      ansible.builtin.apt:
        name: "package"
        install_recommends: no
```

## ansible.builtin.yum

```
---
- name: YUM examples
  hosts: centos

  tasks:
    - name: Install package
      ansible.builtin.yum:
```

```
      name: "nginx"
      state: present

  - name: Install multiple packages
    ansible.builtin.yum:
      name:
        - httpd
        - mod_ssl
        - php
      state: present

  - name: Install specific version
    ansible.builtin.yum:
      name: "kernel-3.10.0"
      state: present

  - name: Remove package
    ansible.builtin.yum:
      name: "oldpackage"
      state: absent

  - name: Update all packages
    ansible.builtin.yum:
      name: "*"
      state: latest

  - name: Install from URL
    ansible.builtin.yum:
      name: "https://example.com/package.rpm"
      state: present

  - name: Enable repo
    ansible.builtin.yum:
      name: "package"
      enablerepo: "epel"

  - name: Disable repo
    ansible.builtin.yum:
      name: "package"
      disablerepo: "testing"
```

## ansible.builtin.dnf

```
---
- name: DNF examples
  hosts: rhel8

  tasks:
    - name: Install package
      ansible.builtin.dnf:
        name: "nginx"
        state: present

    - name: Install list
      ansible.builtin.dnf:
        name:
          - "@Development Tools"
          - python3
          - git
        state: present

    - name: Install group
      ansible.builtin.dnf:
        name: "@Web Server"
        state: present

    - name: Remove package
      ansible.builtin.dnf:
        name: "oldpkg"
        state: absent

    - name: Upgrade all
      ansible.builtin.dnf:
        name: "*"
        state: latest
```

## ansible.builtin.pip

```
---
- name: PIP examples
  hosts: all

  tasks:
    - name: Install Python package
```

```
      ansible.builtin.pip:
        name: "{{ item }}"
        state: present
      loop:
        - django
        - requests
        - boto3

    - name: Install specific version
      ansible.builtin.pip:
        name: "flask==2.0.1"
        state: present

    - name: Install in virtualenv
      ansible.builtin.pip:
        name: "django"
        virtualenv: "/opt/app/venv"
        virtualenv_python: "python3.9"

    - name: Install from requirements
      ansible.builtin.pip:
        requirements: "{{ playbook_dir }}/requirements.txt"
        virtualenv: "/opt/app/venv"

    - name: Install with extra index
      ansible.builtin.pip:
        name: "private-package"
        extra_args: "--index-url https://pypi.example.com/simple"

    - name: Upgrade pip itself
      ansible.builtin.pip:
        name: "pip"
        state: latest

    - name: Install editable package
      ansible.builtin.pip:
        name: "/opt/mypackage"
        editable: yes
        virtualenv: "/opt/venv"
```

## ansible.builtin.git

```
---
- name: Git examples
```

```yaml
hosts: all

tasks:
  - name: Clone repository
    ansible.builtin.git:
      repo: "https://github.com/user/repo.git"
      dest: "/opt/repo"

  - name: Clone specific branch
    ansible.builtin.git:
      repo: "{{ git_repo }}"
      dest: "/opt/app"
      version: "develop"

  - name: Clone specific tag
    ansible.builtin.git:
      repo: "https://github.com/user/repo.git"
      dest: "/opt/app"
      version: "v2.1.0"

  - name: Clone with depth
    ansible.builtin.git:
      repo: "{{ git_repo }}"
      dest: "/opt/app"
      depth: 1

  - name: Update repository
    ansible.builtin.git:
      repo: "{{ git_repo }}"
      dest: "/opt/app"
      update: yes

  - name: Clone with SSH key
    ansible.builtin.git:
      repo: "git@github.com:user/private.git"
      dest: "/opt/private"
      key_file: "/home/user/.ssh/deploy_key"
      accept_hostkey: yes

  - name: Force update
    ansible.builtin.git:
      repo: "{{ git_repo }}"
      dest: "/opt/app"
      force: yes
```

## ansible.builtin.cron

```
---
- name: Cron examples
  hosts: all

  tasks:
    - name: Create cron job
      ansible.builtin.cron:
        name: "Backup database"
        minute: "0"
        hour: "2"
        job: "/usr/local/bin/backup.sh"

    - name: Daily job
      ansible.builtin.cron:
        name: "Daily cleanup"
        special_time: "daily"
        job: "/usr/local/bin/cleanup.sh"

    - name: Weekly job
      ansible.builtin.cron:
        name: "Weekly report"
        special_time: "weekly"
        job: "/usr/local/bin/report.sh"
        user: "{{ app_user }}"

    - name: Complex schedule
      ansible.builtin.cron:
        name: "Complex task"
        minute: "*/15"
        hour: "9-17"
        weekday: "1-5"
        job: "/usr/local/bin/task.sh"

    - name: Cron with environment
      ansible.builtin.cron:
        name: "Job with env"
        minute: "30"
        hour: "3"
        job: "/usr/local/bin/job.sh"
        env: yes
        insertafter: "PATH"

    - name: Remove cron job
```

```
      ansible.builtin.cron:
        name: "Old job"
        state: absent

    - name: Disable cron job
      ansible.builtin.cron:
        name: "Temp disabled"
        minute: "0"
        hour: "4"
        job: "/usr/local/bin/task.sh"
        disabled: yes
```

## ansible.builtin.mount

```
---
- name: Mount examples
  hosts: all

  tasks:
    - name: Mount filesystem
      ansible.builtin.mount:
        path: "/data"
        src: "/dev/sdb1"
        fstype: "ext4"
        state: mounted

    - name: Add to fstab only
      ansible.builtin.mount:
        path: "/backup"
        src: "nfs.example.com:/export/backup"
        fstype: "nfs"
        opts: "defaults"
        state: present

    - name: Mount with options
      ansible.builtin.mount:
        path: "/mnt/share"
        src: "//server/share"
        fstype: "cifs"
        opts: "username={{ smb_user }},password={{ smb_pass }}"
        state: mounted

    - name: Unmount
      ansible.builtin.mount:
```

```
        path: "/old_mount"
        state: unmounted

    - name: Remove from fstab
      ansible.builtin.mount:
        path: "/removed"
        state: absent

    - name: Remount
      ansible.builtin.mount:
        path: "/data"
        state: remounted
```

## ansible.builtin.wait_for

```
---
- name: Wait_for examples
  hosts: all

  tasks:
    - name: Wait for port
      ansible.builtin.wait_for:
        port: 22
        host: "{{ inventory_hostname }}"
        timeout: 300

    - name: Wait for file to exist
      ansible.builtin.wait_for:
        path: "/var/run/app.pid"
        state: present

    - name: Wait for file to be removed
      ansible.builtin.wait_for:
        path: "/tmp/lock"
        state: absent

    - name: Wait for string in file
      ansible.builtin.wait_for:
        path: "/var/log/app.log"
        search_regex: "Application started"

    - name: Wait for service
      ansible.builtin.wait_for:
        port: 80
```

```
        delay: 10
        timeout: 60

    - name: Wait for connection
      ansible.builtin.wait_for_connection:
        timeout: 300
```

## ansible.builtin.uri

```
---
- name: URI examples
  hosts: all

  tasks:
    - name: HTTP GET request
      ansible.builtin.uri:
        url: "https://api.example.com/status"
        method: GET
      register: api_response

    - name: POST with JSON
      ansible.builtin.uri:
        url: "https://api.example.com/data"
        method: POST
        body_format: json
        body:
          key: "{{ value }}"
          name: "{{ item_name }}"
        headers:
          Authorization: "Bearer {{ api_token }}"
      register: post_result

    - name: Check HTTP status
      ansible.builtin.uri:
        url: "http://{{ inventory_hostname }}/health"
        status_code: 200
        validate_certs: no

    - name: Download file
      ansible.builtin.uri:
        url: "https://example.com/file"
        dest: "/tmp/downloaded_file"
        creates: "/tmp/downloaded_file"
```

```
    - name: With basic auth
      ansible.builtin.uri:
        url: "https://secure.example.com/api"
        user: "{{ api_user }}"
        password: "{{ api_pass }}"
        force_basic_auth: yes
```

## ansible.builtin.assert

```
---
- name: Assert examples
  hosts: all

  tasks:
    - name: Assert condition
      ansible.builtin.assert:
        that:
          - ansible_distribution == "Ubuntu"
        fail_msg: "This playbook requires Ubuntu"
        success_msg: "Ubuntu detected"

    - name: Multiple conditions
      ansible.builtin.assert:
        that:
          - ansible_memtotal_mb >= 2048
          - ansible_processor_cores >= 2
        fail_msg: "System does not meet minimum requirements"

    - name: Check variable defined
      ansible.builtin.assert:
        that:
          - app_name is defined
          - db_host is defined
        fail_msg: "Required variables not defined"
```

## ansible.builtin.fail

```
---
- name: Fail examples
  hosts: all

  tasks:
```

```
    - name: Fail with message
      ansible.builtin.fail:
        msg: "Custom failure message"
      when: some_condition

    - name: Conditional failure
      ansible.builtin.fail:
        msg: "Disk space too low: {{ ansible_mounts[0].size_available }}"
      when: ansible_mounts[0].size_available < 1000000000
```

## ansible.builtin.pause

```
---
- name: Pause examples
  hosts: all

  tasks:
    - name: Pause for confirmation
      ansible.builtin.pause:
        prompt: "Press Enter to continue or Ctrl+C to abort"

    - name: Pause with seconds
      ansible.builtin.pause:
        seconds: 30

    - name: Pause with minutes
      ansible.builtin.pause:
        minutes: 5

    - name: Pause for user input
      ansible.builtin.pause:
        prompt: "Enter the deployment environment"
      register: deployment_env

    - name: Use input
      ansible.builtin.debug:
        msg: "Deploying to {{ deployment_env.user_input }}"
```

## ansible.builtin.include_vars

```
---
- name: Include_vars examples
```

```
  hosts: all

  tasks:
    - name: Include variable file
      ansible.builtin.include_vars:
        file: "{{ playbook_dir }}/vars/{{ ansible_distribution }}.yml"

    - name: Include with dir
      ansible.builtin.include_vars:
        dir: "{{ playbook_dir }}/vars"
        extensions:
          - "yml"
          - "yaml"

    - name: Include with name
      ansible.builtin.include_vars:
        file: "secrets.yml"
        name: "secrets"

    - name: Use loaded vars
      ansible.builtin.debug:
        msg: "{{ secrets.api_key }}"
```

## ansible.builtin.add_host

```
---
- name: Add_host examples
  hosts: localhost

  tasks:
    - name: Add host dynamically
      ansible.builtin.add_host:
        name: "{{ item }}"
        groups: "dynamic_group"
        ansible_host: "{{ item }}"
      loop:
        - "192.168.1.10"
        - "192.168.1.11"

    - name: Add with variables
      ansible.builtin.add_host:
        name: "newserver"
        ansible_host: "10.0.0.50"
        ansible_user: "admin"
```

```
        custom_var: "value"

- name: Use dynamic hosts
  hosts: dynamic_group
  tasks:
    - name: Ping new hosts
      ansible.builtin.ping:
```

## ansible.builtin.meta

```
---
- name: Meta examples
  hosts: all

  tasks:
    - name: Trigger handlers
      ansible.builtin.command: "touch /tmp/file"
      notify: restart_service

    - name: Flush handlers now
      ansible.builtin.meta: flush_handlers

    - name: Refresh inventory
      ansible.builtin.meta: refresh_inventory

    - name: Clear facts
      ansible.builtin.meta: clear_facts

    - name: Clear host errors
      ansible.builtin.meta: clear_host_errors

    - name: End play
      ansible.builtin.meta: end_play

    - name: End host
      ansible.builtin.meta: end_host

  handlers:
    - name: restart_service
      ansible.builtin.service:
        name: myservice
        state: restarted
```

---

## Advanced Variable Patterns

### Nested Loops with Variables

```
---
- name: Nested loop examples
  hosts: all
  vars:
    users:
      - name: "alice"
        databases:
          - "app_db"
          - "test_db"
      - name: "bob"
        databases:
          - "analytics_db"

  tasks:
    - name: Grant database access
      ansible.builtin.debug:
        msg: "Grant {{ item.0.name }} access to {{ item.1 }}"
      loop: "{{ users | subelements('databases') }}"
```

### Dynamic Variable Names

```
---
- name: Dynamic variable names
  hosts: all
  vars:
    env: "production"
    production_db: "prod.db.com"
    staging_db: "stage.db.com"

  tasks:
    - name: Use dynamic variable
      ansible.builtin.debug:
        msg: "{{ vars[env + '_db'] }}"

    - name: Set dynamic variable
      ansible.builtin.set_fact:
        "{{ env }}_port": 5432
```

## Complex Conditionals

```
---
- name: Complex conditionals
  hosts: all

  tasks:
    - name: Multiple conditions
      ansible.builtin.debug:
        msg: "Conditions met"
      when:
        - ansible_distribution == "Ubuntu"
        - ansible_distribution_major_version | int >= 20
        - ansible_memtotal_mb >= 4096

    - name: OR conditions
      ansible.builtin.debug:
        msg: "One condition met"
      when: >
        ansible_distribution == "Ubuntu" or
        ansible_distribution == "Debian"

    - name: Variable in list
      ansible.builtin.debug:
        msg: "In list"
      when: inventory_hostname in groups['webservers']
```

---

**Last Updated:** February 2026
**Ansible Version:** 2.15+